Q&A Notes: Code Change Risk Predictor

Short answers to the questions people are most likely to ask about the project, plus the numbers worth remembering and a plan for the live demo.

Numbers to remember

What	Number
Commits in the dataset	2,788 (psf/requests, Feb 2011 to Aug 2025)
Buggy share	21.9% (about 1 buggy for every 3.6 clean)
Bug rate, train vs test	23.6% vs 15.0%
Train / test split	oldest 80% (2,230) / newest 20% (558)
Algorithms compared	13
Best model	tuned XGBoost, 108 trees of depth 2
CV ROC-AUC (train)	0.732
Test ROC-AUC	0.779
Test PR-AUC	0.401 (random would get 0.150)
Recall at the chosen threshold	0.80 (67 of 84 buggy commits caught)
Bugs found in the riskiest 20% of commits	48%, which is 2.4 times random
Top feature	lines added (about 11 points of risk on average)
SHAP and LIME agreement	60% overlap in the top 3 features
Tests in the repo	51

The problem

What does your project do, in one sentence? It looks at a git commit, estimates how likely it is to introduce a bug, and explains the reasons in plain English so a reviewer knows what to look at.

Why is this useful? Reviewers have limited time. A big refactor and a typo fix usually get the same attention. If we can rank commits by risk, reviewers can spend their time on the few commits that matter. In our test set, reviewing the top 20% finds about half of the buggy commits.

What does "just-in-time" mean? The prediction is made at commit time, before the change is merged, while the developer still remembers what they did. Older defect prediction worked at the file or module level after a release, which is too late to be practical.

Data and labels

Where does the data come from? The complete git history of psf/requests, a popular Python HTTP library. We mined it with PyDriller and kept only non-merge commits that change code.

How do you know which commits were buggy? With the SZZ algorithm. We find commits whose message says they fix a bug, then use `git blame` on the lines each fix changed. Whichever commit last touched those lines is labelled as bug-introducing.

Is SZZ reliable? Not fully, and it's important to say so. It depends on commit messages, so it misses some fixes and counts some non-fixes. Fixes that only add lines can't be traced. Sometimes a commit that just moved buggy code gets blamed. One example in our data is "Add automatic code formatting", which got labelled buggy only because later fixes touched lines it reformatted.

Why did you drop the last 180 days? Bugs in very recent commits usually haven't been found and fixed yet, so those commits would look clean even if they aren't.

Why did you exclude the word "patch" from the fix keywords? Because requests is an HTTP library, and "PATCH" there usually means the HTTP method, not a bug fix.

Features

What are the 14 features? They come from Kamei et al. (2013) and fall into five groups. Diffusion: subsystems, directories and files touched, plus entropy. Size: lines added, lines deleted and file size before the change. Purpose: whether it is a fix. History: past developers on the files, days since the files last changed, and past changes to the files. Experience: the author's total, recent and subsystem experience.

What is entropy here? It measures how evenly the changed lines are spread across the files. If all changes are in one file it is 0. If they are spread evenly it is 1. Scattered changes are harder to review and tend to be riskier (Hassan, 2009).

How do you avoid leaking future information into the features? We walk the history from oldest to newest. History and experience features for a commit are calculated only from the commits before it, and then we update the running totals.

Evaluation

Why not a normal random train/test split? A random split would let the model train on commits from 2020 and then be tested on commits from 2012. That can't happen in real life, and it makes results look better than they are. We train on the oldest 80% and test on the newest 20%.

How did you choose the best model? By mean ROC-AUC in 5-fold `TimeSeriesSplit` cross-validation inside the training period. The test set wasn't used for choosing anything, not even the threshold.

Why ROC-AUC and PR-AUC instead of accuracy? Only 22% of commits are buggy. A model that always says "clean" would be 78% accurate and useless. ROC-AUC measures how well the model ranks buggy above clean. PR-AUC focuses on the buggy class, and its random baseline equals the bug rate (0.150 on our test set).

What is MCC? Matthews correlation coefficient. It is a correlation between predictions and truth that uses all four cells of the confusion matrix, so it stays honest on imbalanced data. It ranges from -1 to 1.

What is Recall@20% effort and why is yours low? It counts how many buggy commits you'd catch if you reviewed the riskiest commits first and stopped after reading 20% of all changed lines. Ours is low (0.083) because the model ranks big commits as risky, and big commits use up a line-based budget fast. Size is both the strongest risk signal and the thing that makes review expensive.

Why are test scores higher than CV scores? The later history of requests is calmer, and its big risky commits are easier to spot. It isn't leakage, because the test period was never used in training or selection.

Models

Why did XGBoost win? It had the best cross-validated ROC-AUC (0.732). It also handles skewed features and correlated features well, and its explanations all point in sensible directions. That said, the top seven models are within one standard deviation of each other, so it's a narrow win.

Why is a single decision tree so much worse? It overfits. Our complexity curve shows that at depth 15 the tree scores 0.997 on its training folds but only 0.518 on validation. Ensembles fix this by averaging many trees (bagging) or adding many shallow ones (boosting).

Explain bias and variance using your results. A depth-1 tree is too simple (high bias), a depth-15 tree memorises the training data (high variance). Validation ROC-AUC peaks in between. The learning curve for XGBoost shows training and validation scores converging around 0.75 to 0.76, so variance is under control.

What did hyperparameter tuning do? It helped the flexible models most. XGBoost went from 0.673 to 0.732 CV ROC-AUC, mostly by moving from depth-6 trees to depth-2 trees with more regularisation.

What is the One-Class SVM doing in the table? It's an anomaly detector trained only on clean commits. It still beats random (test ROC-AUC 0.662), which shows that unusual commits tend to be risky. It can't be chosen as the final model because it never learns what a buggy commit looks like.

Why didn't the perceptron work? It is a linear classifier without probabilities, and the classes aren't linearly separable here. Its test ROC-AUC was only 0.523, close to random.

Class imbalance

How did you handle the imbalance? We compared no handling, class weights and SMOTE. The final model uses class weights (`scale_pos_weight` in XGBoost).

Which worked best? They ranked commits about equally well (ROC-AUC within 0.015). The big difference is at the default 0.5 threshold. Without rebalancing, logistic regression caught only 31% of buggy commits, and with rebalancing it caught 74%. Once we tune the threshold on validation data, all three end up close.

Why does SMOTE only run on the training folds? If synthetic samples ended up in validation or test data, we'd be testing on made-up commits, and the scores would be meaningless.

Unsupervised analysis

What did clustering show? K-Means with k = 3 found a group of cross-cutting commits (touching several subsystems and files) that are buggy 40.7% of the time, about twice the other two groups. The clustering never saw the labels.

What did PCA and t-SNE show? Buggy and clean commits overlap heavily in 2D. PCA needs 5 components for 80% of the variance. This told us early on that no model would separate the classes perfectly.

Explainability

What is SHAP? It splits a prediction into contributions from each feature, based on Shapley values from game theory. The average risk plus all the contributions adds up exactly to the commit's predicted risk.

What is LIME and how does it differ? LIME perturbs the commit, sees how the prediction changes, and fits a small linear model around it. It's model-agnostic like SHAP but depends on random sampling, so results can vary between runs. Our two methods agreed on 60% of the top 3 features.

What problem did you find with explanations? With logistic regression, some explanations were backwards, like "touching more files lowers the risk". This came from highly correlated features (files touched and entropy have a correlation of 0.97) getting opposite weights that cancel out. Accuracy was fine but the explanations weren't trustworthy. XGBoost didn't have this problem.

What is the most important feature? Lines added, by a wide margin. On average it moves the predicted risk by about 11 percentage points, more than all the other features combined.

Application and limitations

How would this be used for real? As a bot on pull requests or a pre-merge check. It scores each new commit, posts the risk and the top reasons, and reviewers look harder at the high-risk ones. Our command-line tool already does the scoring part.

What are the main limitations? Noisy SZZ labels, data from one project only, drift over time (the bug rate fell from 23.6% to 15.0%), and probabilities that are shifted upwards by class weighting, so they should be read as relative risk.

What would you do next? Try training on one project and testing on another, use a cleaner SZZ variant, add features from the diff content itself, and make scoring incremental so a git hook can score a commit in under a second.

Demo plan (about 3 minutes)

1. Start the app with `uv run streamlit run app/streamlit_app.py`.
2. **Overview.** Point at the bug rate chart and mention that it drops over time, which is why we split by time.
3. **Model Leaderboard.** Show that XGBoost was picked by cross-validation, and that the top models are close.
4. **Explain a Commit.** Pick the top commit, read out the first reason, and show the SHAP chart and the real SZZ label underneath.
5. **What-If Simulator.** Halve the lines added and files touched, and show the risk going down. Then press "Suggest how to lower risk".
6. If there's time, run `uv run riskpredictor top -n 5` in a terminal to show the same model works from the command line.